



EDA 课程设计报告

学 院： 自动化学院

班 级： 电信 2303 班

指导教师： 张俊

学 号： 8207230322

姓 名： 樊博文

目录

一、课程设计内容及要求.....	2
1. 课程设计目的.....	2
2. 课程设计要求.....	2
二、原理及方法.....	3
1. 设计概述与基本思想.....	3
2. 指令系统与编码格式.....	4
3. 体系结构与模块划分.....	10
4. 数据通路与控制通路.....	16
数据通路.....	16
控制电路.....	17
5. 五级流水线原理.....	18
6. 冒险处理机制.....	24
三、代码设计.....	27
1. 代码修改.....	27
2. 代码解释.....	42
PC 更新选择逻辑.....	42
取指流水线冲刷.....	43
写回数据选择.....	43
DC 阶段数据旁路选择.....	44
EX 阶段数据旁路选择.....	45
四、测试环境、测试结果及分析.....	46
1. Modelsim 仿真.....	46
仿真平台.....	46
步骤.....	46
仿真结果.....	47
波形分析.....	47
2. FPGA 测试 (RISC16+UART 通信).....	49
RISC16+UART 通信平台.....	49
程序流程图.....	49
测试步骤.....	50
测试结果与分析.....	50
五、总结.....	51

一、课程设计及要求

1.课程设计目的

掌握硬件描述语言 Verilog HDL 的语法及设计技巧，了解 Quartus II 软件的应用，学习 Quartus II 环境下设计 CPU 的基本过程；掌握 CPU 设计代码的含义以及 CPU 的工作原理；了解 CPU 与内存 RAM 间的连接数据的传输过程；学习硬件流水线的工作原理及设计技巧；掌握 RTL 代码的仿真验证技术。融会贯通课堂讲授内容，通过知识的综合运用，加深对计算机系统各模块的工作原理及相互联系的认识。学习设计和调试计算机的基本步骤和 方法，提高使用软件仿真工具和集成电路的基本技能。培养科学研究的独立工作能力，取得工程设计与组装调试的实践和经验。

2.课程设计要求

1. 理解指令集定义，进行硬件体系结构设计，画出 RISC 处理器的体系结构图（指令集见 instruction set.doc）。
2. 使用硬件描述语言 Verilog HDL 完成 RISC 处理器的 RTL 设计。
3. 参考给出的 RISC_test.v，搭建测试模块；为每条指令编写基本测试代码，转换为二进制文件，利用 ModelSim 仿真验证是否每条指令都能正确执行；编写完整的测试代码（确保包含所有指令），验证处理器的正确执行。
4. QuartusII 中建立工程，加入 RISC 处理器、UART 模块、指令存储器、数据存储器及总线模块进行综合编译，确认 RISC 处理器的最大工作主频，并连接 UART 电缆，利用串口通信测试 RISC 处理器是否能正常工作。
5. 课程设计分组进行，各自提交课程设计报告。

二、原理及方法

1.设计概述与基本思想

本课程设计实现的处理器采用精简指令集（RISC）思想：以较少且规则的指令类型完成常用算术逻辑运算、数据访存与控制转移操作，从而降低译码与控制逻辑复杂度，便于在FPGA上进行模块化实现与时序优化。CPU整体采用32位数据通路，内部设置32个32位通用寄存器（GPR0~GPR31），支持寄存器-寄存器运算、寄存器-立即数写入、以及基于寄存器的数据存取。指令系统采用16位定长格式，指令高6位Inst[15:10]作为操作码（Opcode），其余位用于寄存器编号、立即数或位移/地址等字段。定长指令与统一的字段划分使得译码过程能够以操作码驱动控制信号的方式完成，控制器仅需根据Opcode产生ALU操作码、写回控制、访存控制以及分支跳转控制等信号即可。

为提高吞吐率，处理器采用五级流水线结构，将一条指令的执行过程拆分为多个相对独立的阶段，并在阶段之间插入流水寄存器以隔离组合逻辑、缩短关键路径，从而使多条指令在同一时刻处于不同阶段并行推进。流水线阶段划分为IA（指令地址计算）、IF（取指）、DC（译码/读寄存器/生成控制）、EX（执行/访存相关处理）、WB（写回）。其中，IA阶段根据PC选择信号计算下一条指令地址；IF阶段将指令地址输出到指令存储器接口以完成取指；DC阶段对指令进行译码并从寄存器堆读出操作数，同时生成后续阶段需要的控制信号；EX阶段完成ALU运算、移位、比较与标志位更新，并形成访存地址与写回数据；WB阶段把结果写入目标寄存器。为了增强数据前递能力、进一步缓解流水线数据相关带来的停顿，设计在写回信息之后增加了一级暂存寄存器（WBXX），使得最近两条产生结果的指令都能够通过旁路通路被后续指令直接使用，从而减少等待写回造成的等待。

在PC更新机制方面，处理器同时支持顺序执行、相对分支与绝对跳转三种典型形式。顺序执行时PC按半字对齐递增，即每次 $PC = PC + 2$ ；相对分支时采用指令低10位作为位移量，进行符号扩展后左移1位（等效乘2）并与当前PC相加形成目标地址；绝对跳转时将低10位地址左移1位后与高位补零拼接形成目标PC。复位采用低电平有效复位信号Rstn，并配合复位标志实现复位后PC跳转至预设的复位地址Rstn_Addr，从而保证处理器从指定入口开始取指执行。

在控制流方面，译码阶段根据Opcode确定指令是否为条件分支或无条件跳转，并给出分支/跳转类型编码；分支判定依赖ALU产生并保存在标志寄存器中的N/Z状态，其中Z表

示结果为 0, N 表示结果为负。条件分支类指令根据 N/Z 组合判断是否成立, 从而决定 PC 选择信号 PC_Sel 取“分支目标”或“顺序下一条”; 跳转指令则直接令 PC_Sel 选择“跳转目标”。由于流水线在分支判定之前已经取到了顺序下一条指令, 为避免控制流改变时错误指令继续向后推进, 设计在检测到发生分支/跳转(即 PC 选择不为顺序)时, 对 IF/DC 流水寄存器执行冲刷(Flush), 将其内容清零, 使得错误路径上的指令被及时丢弃, 从而保证执行路径正确性。

在数据通路与存储器接口方面, 处理器提供与数据存储器交互的标准信号: DW 作为写使能, DAddr 作为 32 位数据地址, WData 作为 32 位写数据, RData 作为 32 位读数据输入。对 LD(加载)指令, EX 阶段选择 RData 作为写回数据并随流水寄存器传递到写回阶段写入目标寄存器; 对 ST(存储)指令, 译码阶段产生 DW 使能, 并给出写地址与写数据。由于 ST 可能需要使用前序指令刚刚产生、但尚未写回寄存器堆的最新数据, 设计在访存地址与写数据路径上同样加入旁路选择, 确保在存在数据相关时仍能写入正确的数据。

为解决流水线中的数据冒险, 处理器采用旁路(bypass)机制: 当 EX 阶段或 DC 阶段需要使用的源寄存器与前序指令的目的寄存器相同, 且前序指令将写回结果时, 旁路控制器通过比较寄存器地址自动选择来自 EXWB 或 WBXX 的最新结果作为 ALU 输入或访存输入, 从而避免等待寄存器堆写回。该机制能够显著减少因 RAW 相关导致的停顿, 提高流水线连续发射与执行的效率。

此外, 处理器包含低功耗等待指令 WAIT。WAIT 的基本思想是冻结前端流水推进: 通过关闭 PC 寄存器写使能与 IF/DC 流水寄存器写使能, 使 PC 保持不变、指令不再更新, 从而达到停机等待的控制效果。在外部条件满足或复位后再恢复推进, 可保证处理器状态稳定、逻辑行为可控。整体而言, 本设计以定长指令 + 分阶段流水 + 旁路转发 + 控制流冲刷为核心思路, 在保证硬件结构清晰、控制逻辑可实现的前提下, 提高了处理器的运行效率与工程可实现性。

2. 指令系统与编码格式

Operand Type 操作数类型

GPR0-31: General Purpose Register (通用寄存器) 0~31

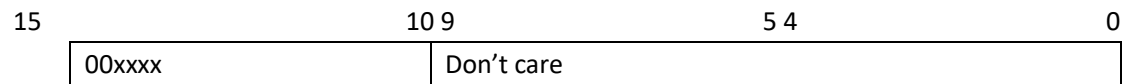
imm#5: 5bit 无符号立即数, 运算时高位补 0, 扩展为 32 位。

rel#10: 10bit 有符号偏移地址，对应半字地址。

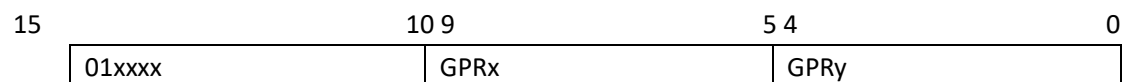
dir#10: 10bit 无符号绝对地址，对应半字地址。

Instruction Type 指令类型

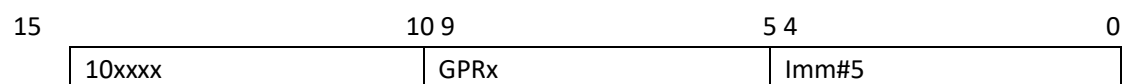
Type 0: (无操作数)



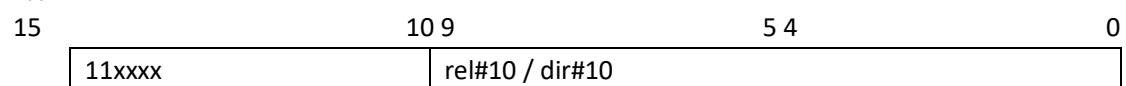
Type 1: (两个操作数，且都为寄存器)



Type 2: (两个操作数，1 个为寄存器，另一个为立即数)



Type 3: (1 个操作数，为立即数)



指令系统在编码上采用 16 位定长 + 高位统一 Opcode + 低位按类型复用字段的组织方式，使得译码过程可以高度规则化：控制器只需依据 Inst[15:10] 即可直接产生 ALU 控制、写回控制、访存控制以及分支跳转控制信号，降低了硬件译码复杂度，并有利于流水线在 DC 阶段快速稳定地产生控制量。

• 算术与逻辑操作指令：

例：

ADD (ADDition of 32-bit integer data)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
OP1+OP2→OP1	ADD OP1,OP2	GPR0-31	GPR0-31	6'b010000

操作

汇编指令

操作数类型

操作码

Description:

32 位有符号整数加法。

计算结果溢出时，目的寄存器中写入最大的有符号整数（最大正整数：0x7fffffff 或最大的负

整数: 0x80000000) .

Operation:

OP1+OP2→OP1;

PC+2 -> PC; PC (程序计数器)

Flag:

N: 计算结果为负时为 1, 否则为 0;

Z: 计算结果为 0 时为 1, 否则为 0;

//TO: 计算结果溢出时为 1,并保持不变;

例:

ADD r0 r1 010000_00000_00001

ADD r2 r0 010000_00010_00000

OR (OR of 32-bit integer data by each bit)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
OP1 or OP2→OP1	OR OP1,OP2	GPR0-31	GPR0-31	6'b010101

Description:

32 位 2 进制数的逻辑或运算。

Operation:

OP1 or OP2→OP1;

PC+2 -> PC;

Flag:

N: 计算结果为负时为 1, 否则为 0;

Z: 计算结果为 0 时为 1, 否则为 0;

例:

OR r2 r3 010101_00010_00011

- 加载/存储指令

LD (Load word)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
(OP2) → OP1	LD OP1,OP2	GPR0-31	GPR0-31	6'b011000

Description:

内存读取操作。将第二个操作数的寄存器数值作为地址，从内存中读取 32 位数据保存如第一个操作数的寄存器。

Operation:

(OP2)→OP1;

PC+2 -> PC;

Flag:

无变化

例:

LD r2 r3 011000_00010_00011

- 分支指令

Bcc (conditional Branch)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
Condition is true :	BZ rel#10	rel#10	—	6'b110000
PC+2+rel#10<<1→PC	BNZ rel#10	rel#10	—	6'b110001
Condition is false :	BLE rel#10	rel#10	—	6'b110010
No operation	BLT rel#10	rel#10	—	6'b110011
	BGE rel#10	rel#10	—	6'b110100
	BGT rel#10	rel#10	—	6'b110101
	B rel#10	rel#10	—	6'b110110

Condition of Branch

Instruction	condition	meaning
-------------	-----------	---------

BZ	Z==1	Branch if zero
BNZ	Z==0	Branch if not zero
BLE	Z==1 or N==1	Branch if Less or Equal
BLT	Z==0 and N==1	Branch if less
BGE	Z==1 or N==0	Branch if Greater or Equal
BGT	Z==0 and N==0	Branch if Greater
B	None	Non-conditional Branch

Description:

条件跳转操作。

条件成立时，跳转目的地址由当前 PC 寄存器（指向下一条指令）及偏移地址（10 位操作数）计算所得。如果条件不成立，执行下一条指令。

Operation:

If (condition)

$PC + 2 + rel\#10 \ll 1 \rightarrow PC$

else

$PC + 2 \rightarrow PC$

Flag:

无变化

例：

BZ 4 110000_00000_00100

BZ -4 110000_11111_11100

• 跳转指令

JUMP (JUMP operation)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
OP1 《1 -> PC	JUMP OP1	dir#10	—	6'b111000

Description:

无条件跳转指令。

10 位操作数逻辑左移 1 位作为绝对地址。

Operation:

OP1 《1 -> PC

Flag:

无变化。

例:

JUMP 4 111000_00000_00100

- 等待指令

WAIT (WAIT operation)

Operation	Assembler	Operand Type		OpCode
		OP1	OP2	
Wait Operation	WAIT	—	—	6'b000001

Description:

不进行任何处理，PC 寄存器保持不变。

Operation:

PC 寄存器保持不变。

Flag:

无变化。

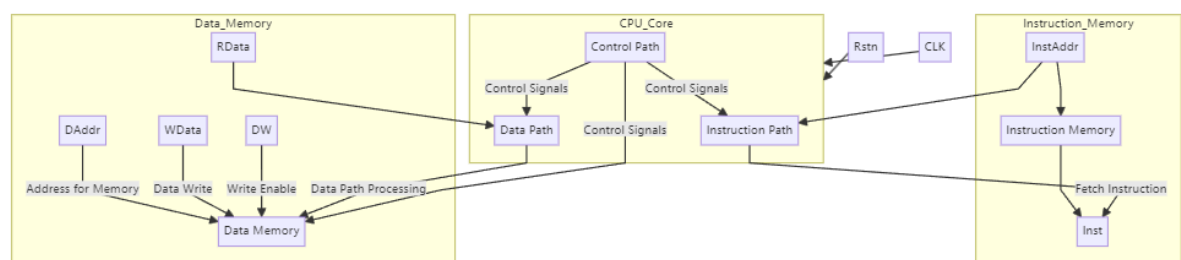
例:

WAIT 000001_00000_00100

3.体系结构与模块划分

3.1 体系结构

本设计处理器采用指令通路、数据通路、控制通路相结合的总体体系结构，面向 16 位定长指令与 32 位数据通路实现一套可流水化执行的 RISC16 CPU。处理器对外提供两类存储器接口：指令存储器用于取指，数据存储器用于 LD/ST 访存；同时以时钟 CLK 驱动各级流水寄存器同步推进，以低电平有效复位 Rstn 保证上电后从确定入口开始执行程序。



外部接口与系统边界

处理器外部可视为 CPU 内核 + 指令存储器 + 数据存储器的组合。指令侧接口以 InstAddr 作为 32 位指令地址输出，指令存储器据此返回 16 位指令 Inst；数据侧接口以 DAddr 作为 32 位数据地址输出，WData 作为 32 位写数据输出，DW 作为写使能输出，数据存储器在读操作时返回 32 位 RData，在写操作时在 DW 有效周期完成写入。指令与数据接口分离使取指与访存不必争用同一端口，结构上避免了典型结构冒险，有利于流水线持续推进。

总体执行组织与五级流水划分

CPU 内核按 IA、IF、DC、EX、WB 五级流水组织执行，并在写回侧额外设置一级 WBXX 暂存寄存器增强旁路覆盖范围。流水线的核心思想是把一条指令的处理过程拆分为若干相对独立的阶段，并在阶段边界插入寄存器锁存关键数据与控制信号，使多条指令能够在不同阶段并行工作，提升吞吐率。阶段边界对应的流水寄存器通常包括：IFDC（锁存取到的指令）、DCEX（锁存译码控制与源操作数）、EXWB（锁存写回三要素）、WBXX（对写回信息再暂存一级用于旁路）。通过这些寄存器隔离组合逻辑，可以缩短关键路径并提高时钟频率的可达上限。

指令通路

流水线体系结构中控制流的核心在于 PC 更新与分支/跳转处理。本设计支持顺序执行、条件分支与无条件跳转三类控制流形式：顺序执行按固定步长推进；分支基于条件判定决定是否转移；跳转直接改变执行地址。由于流水线在控制流判定完全确定前会提前取到顺序下一条指令，当分支成立或发生跳转时，该已取指令属于错误路径。为保证执行路径正确，体系结构引入冲刷机制：当检测到控制流改变时，对 IF/DC 阶段边界的指令寄存器进行清空处理，使错误路径指令不进入后续阶段。该机制以较小的代价换取控制流正确性，是流水线处理器常用且实现简洁的控制冒险处理方式。

数据相关处理与旁路机制

流水线并行推进会带来指令间数据相关问题，最典型的是后一条指令需要使用前一条指令刚产生但尚未写回寄存器堆的结果。为避免频繁停顿等待，本设计在体系结构上引入旁路前递机制：将前序指令的最新结果在写回前直接提供给后续指令使用，从而降低等待写回造成的性能损失。前递数据来源覆盖写回阶段与其后一拍的 WBXX 暂存级，使得相邻与隔一条指令的相关场景都能得到较好覆盖。旁路的具体比较策略、选择位置以及对访存写数据等路径的覆盖细节在后续章节中进一步说明。

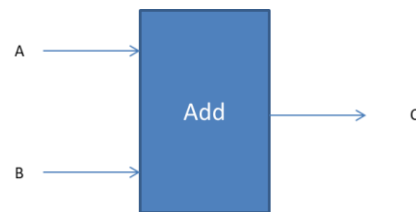
WAIT 冻结机制

为支持等待低功耗控制等需求，体系结构提供 WAIT 指令。其基本行为是冻结前端推进：使程序计数器与取指入口保持不变，从而暂停取指与后续阶段的有效推进，处理器保持当前状态等待外部条件变化或复位恢复。该机制本质上是一种受控的前端停顿，实现简单、行为明确，便于在实验验证中观察处理器暂停与恢复过程。

综上，本设计的体系结构以分离式指令/数据接口 + 五级流水分段 + 控制流冲刷 + 旁路前递 + WAIT 冻结为核心特征。在保证整体结构规则、模块边界清晰的前提下，通过冲刷与旁路两类关键机制分别解决控制流改变与数据相关带来的正确性与效率问题，为后续模块化实现、数据通路控制通路细化分析以及冒险处理验证提供了清晰的架构基础。

3.2 组件模块

- 加法器

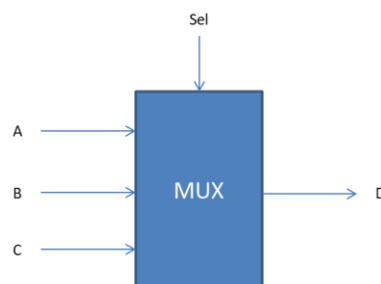


加法器主要服务于 PC 相关地址计算，属于指令通路中的关键组合逻辑。其功能包括：

- PC+2: 用于顺序取指，由于指令为 16 位定长且半字对齐，PC 每次递增 2 字节；
- PC+offset: 用于分支目标地址计算，其中 offset 由指令低 10 位位移量符号扩展后左移 1 位形成字节偏移，再与当前 PC 相加得到分支目标地址。

通过将地址计算集中在加法器模块中，可使 PC 更新逻辑更加规则，且有利于流水线阶段划分与时序收敛。

- 选择器



选择器用于在多种数据来源之间进行选择，是实现多指令功能共用同一数据通路的基础。主要包括：

- PC 选择器: 在 PC+2 (顺序执行)、PC+offset (条件分支目标) 以及跳转绝对地址 ($dir < 1$) 之间选择下一条 PC，最终由 PC_Sel 控制；
- ALU 输入选择器: 在寄存器堆读出的操作数与旁路前递的数据之间进行选择，确保 EX 阶段使用的是最新操作数，减少等待写回造成的停顿；
- 写回数据选择器: 在 ALU 运算结果与 RData (加载指令从数据存储器读出的数据) 之间选择写回来源，保证算术逻辑指令与 LD 指令共享同一写回通路；
- 访存写数据/地址选择器: 对 ST 指令的写地址与写数据进行选择，当源寄存器存在数据相关时，可从前序阶段前递最新值，保证写入数据正确。

总体而言，选择器将同一硬件支持不同指令需求转化为控制信号选择不同路径，使结构简洁且易于扩展。

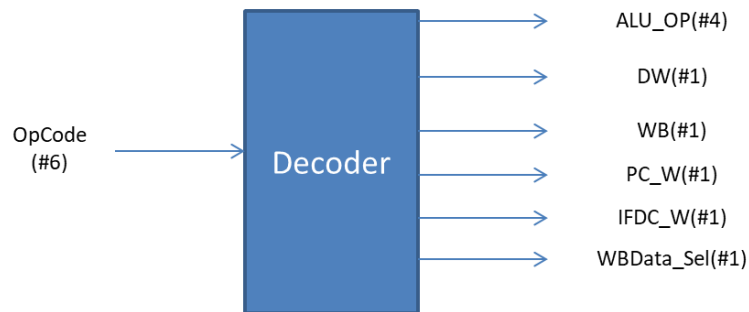
- **流水线寄存器**

用于锁存阶段间信息并隔离组合逻辑，保证流水线同步推进。主要包括：

- IFDC 寄存器：锁存取到的指令；支持写使能与冲刷（Flush）。
- DCEX 寄存器：锁存译码产生的控制信号、源操作数与寄存器地址字段。
- EXWB 寄存器：锁存写回地址、写回数据及写回使能。
- WBXX 寄存器：对写回信息再暂存一级，为旁路提供第二级来源。

PC	PC(#32)
IFDC	Instruction(#16)
DCEX	ALU_OP(#4), DW(#1),WB(#1), WBData_Sel(#1), Reg0(#32),Reg1(#32),Imm(#5),RegAddr0(#5),RegAddr1(#5)
EXWB	WB(#1),WBData(#32),WBAddr(#5)
WBXX	WB(#1),WBData(#32),WBAddr(#5)

- **译码器**



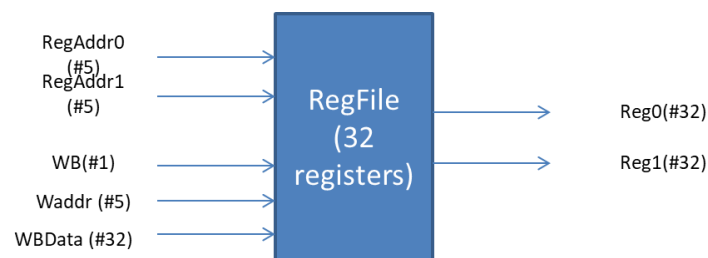
译码器在 DC 阶段对 Opcode 进行组合译码，产生驱动全机的数据与控制信号。主要输出包括：

- ALU 操作控制：决定 EX 阶段 ALU 执行 ADD、SUB/CMP、AND、OR、XOR、移位、立即数生成等功能；
- 写回控制 WB：决定指令是否需要写回寄存器以及写回目标寄存器字段的选择；
- 访存控制 LD/DW：LD 决定写回数据来源为 RData，DW 决定是否对数据存储器执行写操作（ST）；
- 分支/跳转控制 BJ 与 PC 相关控制：根据分支类型与标志位生成 PC_Sel，并在需要时触发 IFDC 冲刷；

— 前端推进控制：如 PC_W、IFDC_W 等，用于实现 WAIT 冻结前端等功能。

译码结果随流水线寄存器向后传递，保证每个阶段在对应周期获得与当前指令匹配的控制信号。

• Register File



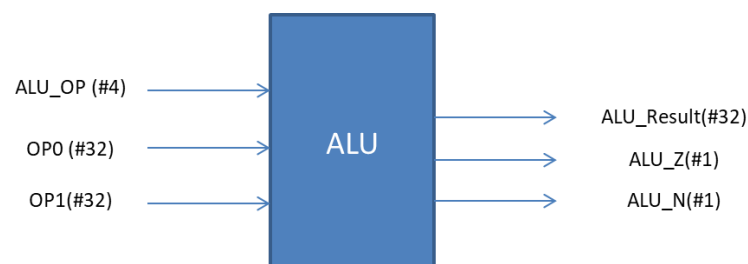
寄存器堆包含 32 个 32 位通用寄存器，提供两个读端口和一个写端口：

(1) 读端口：由指令字段给出寄存器编号，在 DC 阶段读出 Reg0、Reg1 作为运算或访存所需的源操作数；

(2) 写端口：在时钟上升沿、写回使能有效时，将写回数据写入目标寄存器。

寄存器堆是数据通路的中心节点，承担操作数提供、结果保存与数据转发对齐等核心作用。

• ALU



算术逻辑单元，ALU 位于 EX 阶段，完成算术逻辑与移位运算，并产生 N/Z 标志位：

(1) 算术逻辑：ADD、SUB、CMP、AND、OR、XOR 等；

(2) 移位操作：LSL、LSR、ASR 等；

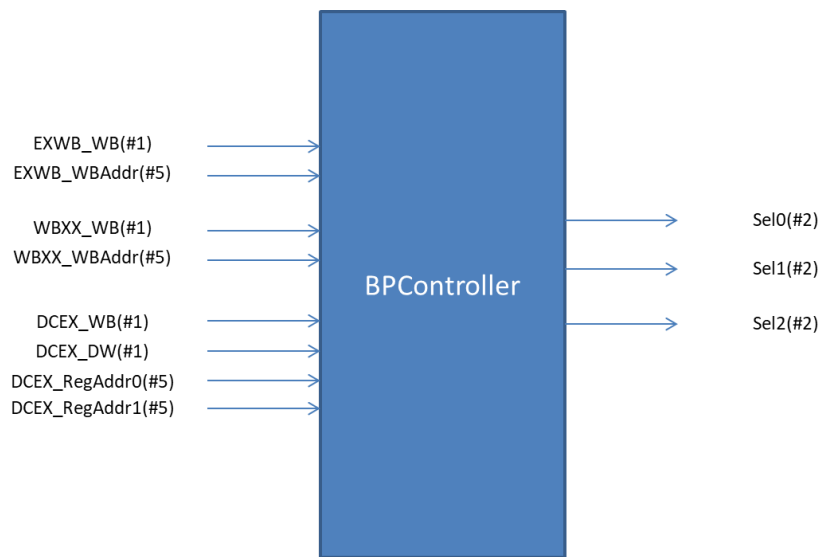
(3) 立即数生成：MOV 将 imm#5 零扩展后形成写回结果；

(4) 标志位：Z 用于表示结果为 0，N 用于表示结果为负，为后续 Bcc 条件判断提供依据。

通过把运算与标志产生集中到 ALU 模块，可使 EX 阶段功能清晰，便于验证与扩展。

- **ByPass Controller**

-



旁路控制器用于解决流水线中的数据相关问题。其基本方法是比较当前指令的源寄存器地址与前序指令的写回地址：若发现匹配且前序指令即将写回，则生成旁路选择信号（如 EX_BP_Sel_0/1、DC_BP_Sel_0/1），将 EXWB 或 WBXX 中的最新结果直接前递到 ALU 输入或访存写数据路径。

该机制可显著减少 RAW 相关导致的停顿，避免因等待写回而插入气泡，从而提高流水线的连续执行能力与整体吞吐率。

4.数据通路与控制通路

数据通路

数据通路是 CPU 完成取指—译码—执行—访存—写回全过程的数据承载与传递载体，决定了指令执行时数据从哪里来、经过哪些部件处理、最终写到哪里去。本设计的数据通路为 32 位宽，通用寄存器、ALU 运算结果、数据存储器读写数据等均以 32 位形式传输，从而保证算术逻辑运算与访存数据宽度统一，结构清晰且便于扩展。

- 数据流向
 - 存储器 Instruction Memory, Data Memory,
 - 寄存器堆栈 Register File,
 - 流水线寄存器 Pipeline Register,
 - 运算器 ALU,
 - 多路选择器 multiplexers.
- 32-bit
-

典型指令的数据通路

- 算术逻辑类（如 ADD、OR）：RegFile 读操作数 → DCEX 锁存 → EX 阶段 ALU 计算 → EXWB 锁存 → WB 写回 RegFile。
- 立即数写入类（MOV）：译码取 imm#5 → EX 阶段扩展形成结果 → EXWB 锁存 → WB 写回 RegFile。
- 加载类（LD）：RegFile 提供地址寄存器 → 形成 DAddr → Data Memory 输出 RData → 写回选择 RData → WB 写入目的寄存器。
- 存储类（ST）：RegFile 提供地址与写数据（必要时旁路）→ 输出 DAddr 与 WData → DW 有效完成写存储器。
- 控制流类（Bcc/JUMP）：译码得到分支类型与位移/地址 → 结合 N/Z 标志产生 PC 选择 → 更新 PC 并冲刷 IFDC，保证控制流正确。

控制电路

本设计采用操作码驱动控制的方式：DC 阶段根据 IFDC_reg[15:10] (Opcode) 进行组合译码，生成包括 ALU 操作类型、写回/访存使能、分支跳转类型等控制量，并通过流水寄存器传递到 EX/WB 阶段以保证控制与数据对齐。该方式结构直观，便于验证，也符合 RISC 处理器规则化译码的特点。

— 从数据通路接收指令码，进行译码产生控制信号；

- 多路选择器的选择
- 寄存器堆栈写信号
- 流水线寄存器写信号
- 流水线寄存器清除信号
- 存储器写信号
- ALU 操作信号
-

在流水线结构下，控制信号必须与对应数据同步前进，否则会造成数据走到 EX 阶段但控制还停在 DC 阶段的错配。本设计在 DC 阶段产生的关键信号（如 DC_ALU_OP、DC_WB、DC_LD、DC_DW 等）通过 DCEX 寄存器锁存并传递到 EX 阶段，再经 EXWB 寄存器与 WBXX 寄存器继续向后传递，确保每一级执行的动作始终与当前指令的数据一致。同时，当控制流发生改变（分支/跳转）时，控制通路通过 PC_Sel 与 IFDC_Flush 快速介入，清除错误取指并重新对齐控制流，使流水线保持正确执行。

总体而言，数据通路负责搬运与处理数据，控制通路负责产生指令行为并指挥各模块协同工作。两者相互配合，使得该 RISC16 CPU 既能在规则化结构下完成算术逻辑、访存和控制转移等核心功能，又能借助旁路与冲刷机制在流水线环境中保持正确性与较高执行效率。

5.五级流水线原理

5 段流水线设计：



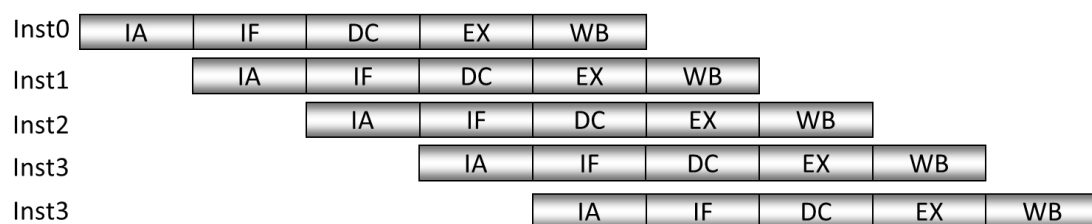
IA : Instruction Address calculation stage

IF : Instruction Fetch stage

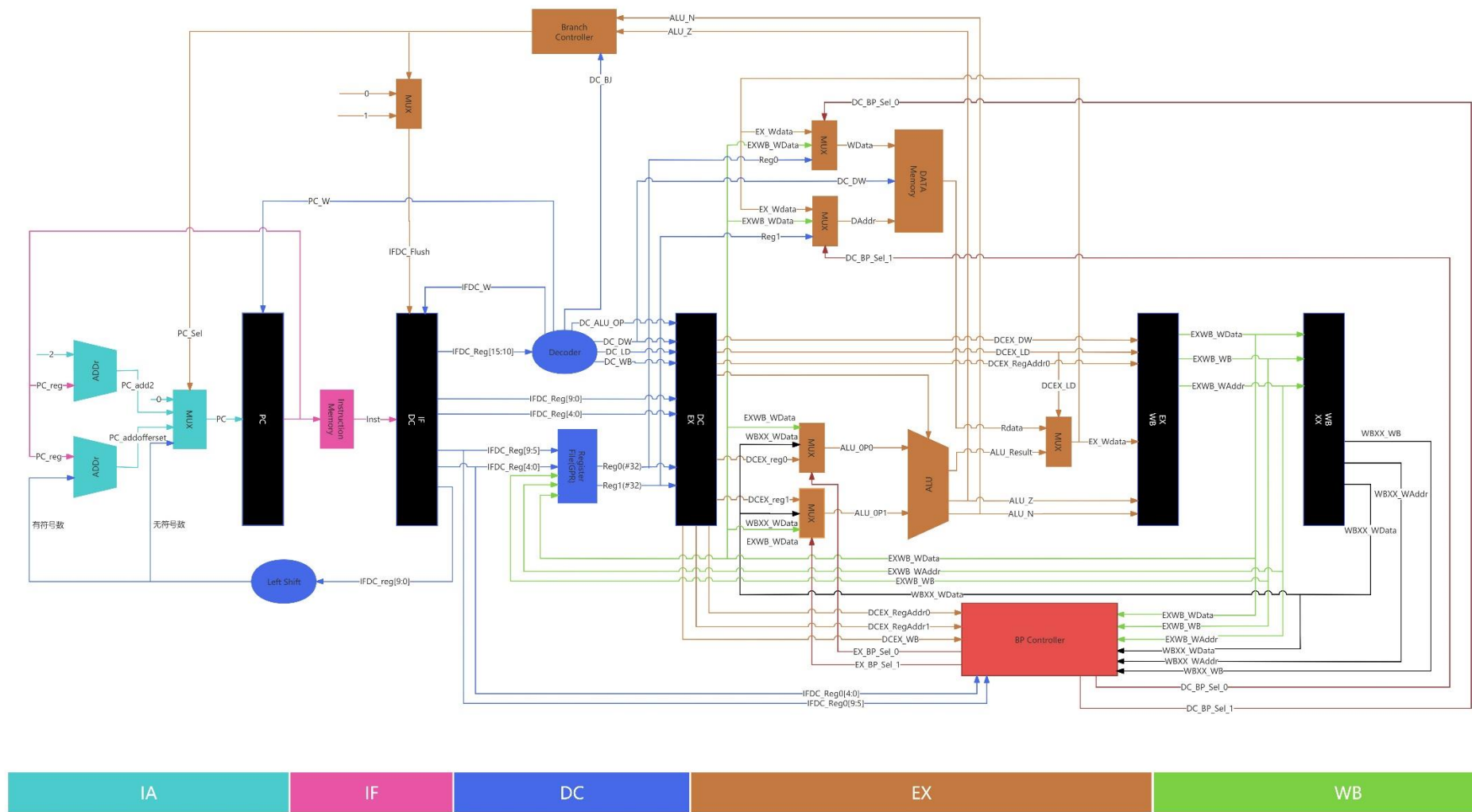
DC : Decode stage

EX : Execution stage

WB : Write Back stage



5 段流水线设计如图所示，处理器将一条指令的执行过程划分为 IA、IF、DC、EX、WB 五个阶段，并通过阶段间流水寄存器实现时序隔离与并行推进。流水线的基本特点是：在同一时钟周期内，不同指令分别处于不同阶段同时工作；当流水线充满后，理想情况下每个时钟周期都能完成一条指令，从而提高整体吞吐率。与单周期/多周期设计相比，流水线把长的组合逻辑路径拆分到多个周期内完成，降低单周期关键路径长度，更有利于提高时钟频率。



一条指令在流水线中依次经历：

IA（产生 PC）→ IF（按 PC 取指）→ IFDC 锁存 → DC（译码/读寄存器/生成控制）→ DCEX 锁存 → EX（ALU/访存/标志位）→ EXWB 锁存 → WB（写回寄存器）→ WBXX 暂存（用于旁路）。

图中还包含两条重要控制回路：

1）分支/跳转回路：由 Branch Controller 结合 DC_BJ 与 ALU_Z/ALU_N 产生 PC_Sel，并触发 IFDC_Flush。

2）旁路回路：由 BP Controller 比较寄存器地址生成 EX_BP_Sel_0/1 与 DC_BP_Sel_0/1，给 ALU 输入与 ST 写数据/地址通路做前递选择。

5.1 IA 阶段（Instruction Address calculation，指令地址计算）

主要目标：计算下一条要取的指令地址 PC，并决定 PC 的来源。

（1）关键部件

a. PC 寄存器 PC_reg：保存当前 PC 值（程序计数器）。

b. 两个加法器：

- PC_add2: $PC_reg + 2$ ，用于顺序执行。
- PC_addoffset: $PC_reg + offset$ ，用于分支目标地址。offset 来自指令低 10 位的符号扩展并左移 1 位（图中 Left Shift 模块）。
- c. PC 选择器（PC_Sel 控制的 MUX）：在“PC+2 / PC+offset / jump 绝对地址”之间选出新的 PC。
- d. PC 写使能 PC_W：决定 PC_reg 是否在本拍更新。WAIT 指令会让 PC_W=0，从而冻结 PC。

（2）PC 的三种典型来源（对应图中 PC_Sel 选择）

a. 顺序执行： $PC = PC_reg + 2$ 。

b. 条件分支： $PC = PC_reg + (signext(rel\#10) \ll 1)$ 。

c. 无条件跳转： $PC = \{高位补 0, dir\#10, 1'b0\}$ （把 10 位绝对地址左移 1 位形成字节地址）。

（3）输出

IA 阶段输出 InstAddr = PC，直接送入指令存储器（Instruction Memory）作为取指地址。

5.2 IF 阶段（Instruction Fetch，取指）

主要目标：根据 InstAddr 从指令存储器读出 16 位指令 Inst，并送入 IFDC 流水寄存器。

（1）关键部件

- a. Instruction Memory：输入 InstAddr，输出 Inst[15:0]。
- b. IFDC 寄存器：把 Inst 锁存为 IFDC_reg，作为 DC 阶段的指令输入。
- c. IFDC 写使能 IFDC_W：控制是否锁存新指令。WAIT 指令会让 IFDC_W=0，从而冻结前端取指/译码入口。
- d. IFDC 冲刷 IFDC_Flush：当发生分支/跳转导致 PC 改变时，将 IFDC 清零，丢弃错误路径已取到的指令（图中从 Branch Controller 回来的 Flush 信号）。

（2）为什么需要 Flush

分支/跳转的判定在后面阶段才明确，但 IF 阶段会“提前”按顺序 PC 取到下一条指令。若控制流改变，这条指令就是错误的，需要通过 IFDC_Flush 把它清掉，避免进入 DC/EX 继续执行。

5.3 DC 阶段（Decode，译码/读寄存器/生成控制）

主要目标：对指令译码产生控制信号；从寄存器堆读取操作数；准备分支/跳转所需的立即数/位移信息；把所有关键数据与控制一起送入 DCEX。

（1）关键部件

- a. Decoder（译码器）：输入 IFDC_reg[15:10]（Opcode），输出控制信号：
 - DC_ALU_OP：ALU 操作码（ADD/SUB/AND/OR/XOR/移位/MOV 等）。
 - DC_WB：是否写回寄存器。
 - DC_LD：是否加载（LD）。
 - DC_DW：是否写数据存储器（ST）。
 - DC_BJ：分支/跳转类型（BZ/BNZ/BLE/.../B/JUMP）。
 - 同时还会决定 PC_W、IFDC_W（例如 WAIT 会关闭二者）。
- b. Register File（寄存器堆）：
 - 读端口：Reg0 = GPR[IFDC_reg[9:5]]，Reg1 = GPR[IFDC_reg[4:0]]。
 - 写端口：在 WB 阶段写回（由 EXWB/WBXX 带来的写回信息驱动）。
- c. 立即数/位移准备：
 - imm#5：来自 IFDC_reg[4:0]，用于 MOV 等。
 - rel#10/dir#10：来自 IFDC_reg[9:0]。

- Left Shift 模块：把 rel#10 左移 1 位，用于形成字节偏移。

d. 分支控制器 Branch Controller:

输入 DC_BJ 与标志位 ALU_Z、ALU_N，输出 PC_Sel 与 IFDC_Flush。

- PC_Sel 决定下一条 PC 取顺序/分支/跳转。
- IFDC_Flush 在 PC 改变时置 1，清除 IF 阶段错取的指令。

(2) DC 阶段输出到下一阶段

DC 阶段把“控制 + 数据”一起送入 DCEX 寄存器锁存：

- 控制：DC_ALU_OP、DC_WB、DC_LD、DC_DW（图中有 DCEX_LD、DCEX_DW 等信号线）。
- 数据：Reg0、Reg1、imm#5、RegAddr0/RegAddr1。

5.4 EX 阶段（Execution，执行/访存/标志位）

主要目标：完成算术逻辑运算与移位/比较；形成访存地址与写数据；选择写回数据来源；更新标志位；并利用旁路解决数据相关。

(1) 关键部件

a. DCEX 寄存器：锁存来自 DC 的操作数与控制信号，保证 EX 阶段处理的是同一条指令对应的数据与控制。

b. ALU 与其输入 MUX:

- ALU_OP0、ALU_OP1 分别来自两路输入选择器。
- 输入选择器受 EX_BP_Sel_0/EX_BP_Sel_1 控制，用于从“DCEX 操作数 / EXWB 结果 / WBXX 结果”中选最新值。
- 这就是图中 ALU 前的两组旁路 MUX。

c. 标志位寄存器 ALU_Z_reg、ALU_N_reg:

算术逻辑/比较运算后更新 Z/N；分支类指令不改数据结果，但会读取这些标志位给 Branch Controller 作条件判断。

d. Data Memory 接口相关路径（图右上棕色 Memory 块与其前的 MUX）:

- DAddr: 通常来自地址寄存器通路（设计里常用 Reg1 作为地址源），并带旁路选择 DC_BP_Sel_1，保证地址相关时也能前递最新值。
- WData: 写数据通路带旁路选择 DC_BP_Sel_0，保证写入的数据是最新值。
- DW: 由 DC_DW 控制，ST 时为 1，其余为 0。

e. 写回数据选择 (EX_WData MUX):

- 若是 LD: 写回数据选 RData (从 Data Memory 读出)。
- 其他运算: 写回数据选 ALU_Result。
- 图中 EX_WData 从“RData/ALU_Result”选择后送入 EXWB。

(2) EX 阶段输出到下一阶段

EX 阶段把写回相关信息送入 EXWB 寄存器:

- EXWB_WData: 要写回的数据 (ALU 结果或 RData)。
- EXWB_WAddr: 目的寄存器地址 (通常来自 DCEX_RegAddr0)。
- EXWB_WB: 写回使能。
- 同时, ALU_Z/ALU_N 更新后回馈到 Branch Controller 用于后续分支判定。

5.5 WB 阶段 (Write Back, 写回)

主要目标: 把结果写入寄存器堆, 完成指令的“架构状态更新”。

(1) 关键部件

- a. EXWB 寄存器: 提供写回三要素 (EXWB_WB、EXWB_WAddr、EXWB_WData)。
- b. Register File 写口: 在时钟上升沿, 若 EXWB_WB=1, 则 $\text{RegFile}[\text{EXWB_WAddr}] \leftarrow \text{EXWB_WData}$ 。

(2) WBXX 暂存级

WBXX 把上一拍的写回信息再锁存一级: WBXX_WB、WBXX_WAddr、WBXX_WData。

它的意义是给旁路提供第二级可前递来源。这样即使后续指令需要的值来自更早一拍完成写回的结果, 也能从 WBXX 直接前递, 提高旁路覆盖范围, 进一步减少 RAW 相关带来的停顿。

图中两类关键控制机制总结

(1) 控制流处理: PC_Sel + IFDC_Flush

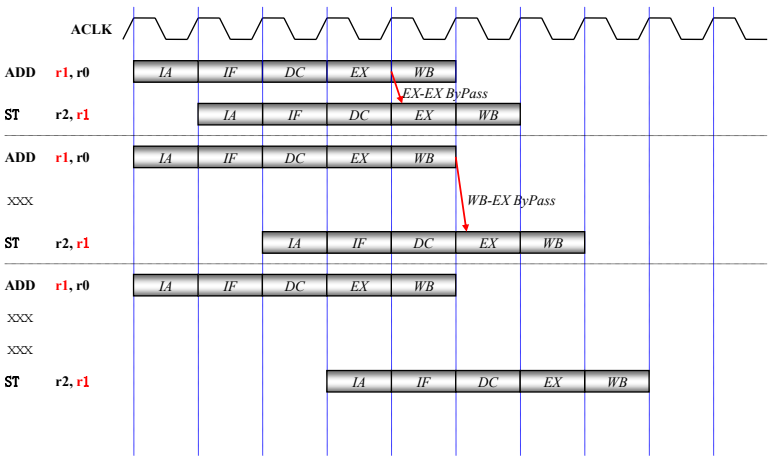
- Branch Controller 用 DC_BJ (分支类型) 和 ALU_Z/ALU_N (标志位) 决定 PC_Sel。
- 只要 PC_Sel 不是顺序执行, 就触发 IFDC_Flush 清除错误取指, 保证控制流正确。

(2) 数据相关处理: BP Controller + 多处旁路 MUX

- BP Controller 比较 DCEX 源寄存器地址与 EXWB/WBXX 的写回地址, 输出:
- EX_BP_Sel_0/1: 给 ALU 输入前递;
- DC_BP_Sel_0/1: 给写数据/地址等路径前递。
- 结果是: 后续指令通常不必等待写回完成就能用到最新数据, 减少气泡, 提高吞吐率。

6.冒险处理机制

流水线处理器在提高吞吐率的同时，会引入冒险问题，即由于指令之间的相关性或硬件资源共享，可能导致某些周期内无法按照理想节拍推进。本设计的总体思路是：本设计的总体处理思路为：数据冒险：以旁路前递（Bypass）为主，减少 RAW 相关造成的停顿；控制冒险：以 PC 选择 + IFDC 冲刷（Flush）纠正取指方向，丢弃错误路径指令。



在仿真波形或时序示意中可观察到：当出现寄存器 RAW 相关时，后续指令通常可以通过 EX→EX 与 WB→EX 两级旁路获得最新结果，从而保持连续推进。

6.1 数据冒险

数据冒险最常见的是 RAW 相关（Read After Write，先写后读）：后一条指令需要读取某寄存器的新值，但该新值由前一条或前几条指令产生，尚未写回寄存器堆。如果不做处理，后一条指令会读到旧值，导致结果错误，因此必须采取停顿插泡或旁路前递来保证正确性。

- 本设计为顺序发射、顺序完成的流水线，重点处理 RAW 即可。典型 RAW 场景包括：
- （1）算术逻辑相关：例如 ADD 的结果作为下一条 SUB/OR 等运算的输入；
 - （2）访存相关：例如前一条指令产生的数据作为 ST 写数据，或产生的值作为 LD/ST 的地址寄存器；
 - （3）标志位相关：前一条运算更新 N/Z 标志位，后一条 Bcc 依赖 N/Z 进行分支判断。

旁路前递机制

旁路前递的核心思想是：不等待结果写回寄存器堆，而是将刚产生的最新结果直接送到后续指令需要使用的位置。结合本设计，旁路主要覆盖两条路径：ALU 输入旁路：用于解决 EX

阶段运算所需操作数的 RAW；数据存储器写数据路径旁路：用于保证 ST 写入的数据为最新值。

ALU 输入旁路

当某条指令进入 EX 阶段需要操作数时，旁路控制通过比较当前指令源寄存器地址与前序指令写回目的寄存器地址，生成选择信号 EX_BP_Sel_0/1，从而在 ALU 输入端选择最新值。代码实现对应如下：

若 EXWB_WB 有效且 EXWB_WAddr == DCEX_RegAddr*，则选择 EXWB_WData (EX→EX)；
否则若 WBXX_WB 有效且 WBXX_WAddr == DCEX_RegAddr*，则选择 WBXX_WData (WB→EX)；
否则使用 DCEX_Reg* (来自寄存器堆读出的原始值)。

对应代码片段（逻辑含义）：

选择信号：EX_BP_Sel_0/EX_BP_Sel_1

数据源：EXWB_WData / WBXX_WData / DCEX_Reg0|Reg1

最终送入：ALU_OP0 / ALU_OP1

因此，即使寄存器堆尚未完成写回，后续运算类指令仍能在 EX 阶段拿到正确操作数，减少因 RAW 引入的气泡，提高流水线连续性。

访存通路旁路

访存类指令中，ST 既需要写地址也需要写数据，这两者都有可能来自前序指令的最新结果。例如：

- （1）写数据相关：ST 要写入的数据寄存器刚由上一条 ADD/OR 等产生；
- （2）地址相关：ST/LD 的地址寄存器刚由上一条指令更新。

因此本设计在数据存储器接口处同样加入旁路选择：

- （1）WData 旁路：当 ST 的写数据寄存器与前序写回寄存器一致时，写数据直接从 EXWB 或 WBXX 前递，避免写入旧值；
- （2）DAddr 旁路：当访存地址寄存器存在相关时，地址也可通过旁路获得最新值，保证访问的是正确地址。

通过把旁路覆盖到 ALU 输入端和存储器接口端,可以解决运算与访存两类常见数据相关,使流水线在多数相关情况下仍保持无停顿推进。

LD 相关说明

对 LD 指令而言,写回数据来自数据存储器输入 RData。在本设计中写回数据在 EX 阶段通过:

$EX_WData = (DCEX_LD) ? RData : ALU_Result$

并进入 EXWB_WData,随后一拍即可被后续指令通过 EXWB/WBXX 前递使用。因此在默认单周期读返回的存储器模型下,LD 通常不需要额外插入停顿。若外部存储器读延迟为多周期,则需要引入冻结/等待机制,但本设计目前以单周期返回为假设,故以旁路为主。

6.2 控制冒险

控制冒险来自分支与跳转指令:取指阶段默认按顺序 PC 取下一条指令,但若分支成立或发生跳转,下一条正确指令应来自分支/跳转目标地址,此时已经取到的顺序指令属于错误路径,必须丢弃。

本设计采用 PC 选择 + IFDC 冲刷的方式处理控制冒险:

(1) PC 选择 PC_Sel: 由分支控制器根据分支类型 DC_BJ 以及标志位 ALU_Z/ALU_N 决定 PC 取值来源(顺序 PC+2、分支目标 PC+offset、跳转目标 dir<<1);

(2) 冲刷 IFDC_Flush: 只要 PC 发生改变(PC_Sel 不为顺序),立即将 IFDC 寄存器清零,丢弃已取到的错误路径指令,保证后续阶段不会执行错误指令。

控制冒险的代价通常表现为分支/跳转带来 1 个周期左右的空转(等价于丢弃一条已取指令)。在本设计中,分支判定依赖 N/Z 标志位,标志位由运算指令产生并锁存;同时也可以理解为对标志位存在一定前递效果,使分支能够尽早完成判定,从而把控制冒险代价控制在较小范围内。

三、代码设计

1.代码修改

在本次课程设计的代码实现过程中，我以老师提供的参考工程为基础进行开发：在充分理解原有框架与数据通路设计思想的前提下，结合指令集要求与自身的实现细节，对部分模块逻辑进行了调整与补全，使其能够覆盖全部指令的译码、流水线传递、旁路控制以及写回流程。

下面给出我整理并注释后的核心代码文件 RISC16.v，其中包含了处理器顶层接口定义、各流水级寄存器与控制信号、指令译码逻辑、分支跳转控制、旁路转发机制、ALU 运算单元以及写回寄存器等完整实现流程，可用于后续仿真验证与串口测试。

```
module RISC16(CLK, Rstn, InstAddr, Inst, DW, DAddr, WData, RData);  
    // 输入信号  
    input          CLK, Rstn;          // CLK: 时钟信号, Rstn: 复位信号 (低电平有效)  
    // 输出信号  
    output [31:0]  InstAddr, DAddr;    // InstAddr: 指令地址, DAddr: 数据地址 (32 位地址)  
    output         DW;                 // DW: 写数据使能信号 (1 位)  
    output [31:0]  WData;              // WData: 写数据 (32 位)  
    // 输入数据  
    input  [15:0]  Inst;               // Inst: 指令输入 (16 位)  
    input  [31:0]  RData;              // RData: 读取数据 (32 位)  
    // 参数定义  
    parameter      Rstn_Addr = 0;     // 参数: 复位地址 (0)  
  
    // IA 阶段  
    reg Rstn_flag;                    // 复位标志  
    reg [31:0] PC_reg;                // 程序计数器寄存器, 存储当前 PC 值  
    wire [31:0] PC_add2, PC_addoffset, PC; // PC + 2, PC + offset, 当前 PC  
    wire [1:0] PC_Sel;                // PC 选择信号, 选择新的 PC 值  
    reg PC_W;                          // 程序计数器写使能信号  
    // DC 阶段  
    reg [15:0] IFDC_reg;              // IF 到 DC 阶段的流水线寄存器, 存储指令  
    reg IFDC_W;                       // IFDC 寄存器写使能  
    wire IFDC_Flush;                  // IFDC 清除信号  
    reg [31:0] RegFile[31:0];         // 寄存器文件, 存储 32 个 32 位寄存器  
    wire [31:0] Reg0, Reg1;           // 操作数 0 和操作数 1, 从寄存器文件读取  
    reg [3:0] DC_ALU_OP;              // ALU 操作码, 控制 ALU 操作
```



```

reg DC_WB, DC_DW, DC_LD; // 写回、存储、加载信号
reg [3:0] DC_BJ; // 分支条件信号
wire [1:0] DC_BP_Sel_0, DC_BP_Sel_1; // 旁路选择信号
// EX 阶段
// reg DCEX_Flush; // 清除 DCEX 寄存器的信号
reg [3:0] DCEX_ALU_OP; // ALU 操作码, 指示 ALU 要执行的操作
reg DCEX_DW, DCEX_LD, DCEX_WB; // 存储、加载、写回信号, 控制数据操作
reg [31:0] DCEX_Reg0, DCEX_Reg1; // 操作数 0 和操作数 1, 传递到 ALU 进行计算
reg [4:0] DCEX_imm, DCEX_RegAddr0, DCEX_RegAddr1; // 立即数、寄存器地址 0 和 1, 传递给 ALU

wire signed [32:0] ALU_OP0, ALU_OP1; // ALU 操作数 0 和 1, 供 ALU 运算使用
reg signed [32:0] ALU_Result; // ALU 计算结果
reg ALU_Z, ALU_N; // ALU 零标志 (Z) 和负标志 (N)
reg ALU_Z_reg, ALU_N_reg; // ALU 零标志和负标志的寄存器, 用于存储 ALU 的状态

wire [1:0] EX_BP_Sel_0, EX_BP_Sel_1; // 旁路选择信号, 用于数据旁路
wire [31:0] EX_WData; // EX 阶段的写回数据
// WB 阶段
reg EXWB_LD; // EXWB 寄存器的加载信号
reg EXWB_WB; // EXWB 寄存器的写回信号
reg [4:0] EXWB_WAddr; // EXWB 寄存器中的写回地址
reg [31:0] EXWB_WData; // EXWB 寄存器中的写回数据
reg [31:0] EXWB_Reg1; // EXWB 寄存器中的操作数 1

// XX stage
reg WBXX_WB; //
writeback signal in WBXX register
reg [4:0] WBXX_WAddr; //
writeback address in WBXX register
reg [31:0] WBXX_WData; //
writeback data in WBXX register

//
// IA stage
//

// reset flag
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn)
        Rstn_flag <= 1'b1;
    else
        Rstn_flag <= 1'b0;
end

```

```

end

assign PC_add2 = PC_reg + 2'b10;
assign PC_addoffset = PC_reg + {{21{!IFDC_reg[9]}},!IFDC_reg[9:0],1'b0};
assign PC = (Rstn_flag == 1'b1) ? Rstn_Addr :
            (PC_Sel == 2'b00) ? PC_add2 :
            (PC_Sel == 2'b01) ? PC_addoffset :
                                     {21'h000000,!IFDC_reg[9:0],1'b0};    // normal case;
branch; jump

//
// IF stage
//

// PC register
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn)
        PC_reg <= 32'h00000000;
    else if (PC_W)
        PC_reg <= PC;
end

// instruction address for InstRAM
assign InstAddr = PC;

//
// DC stage
//

// IFDC register
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn)
        IFDC_reg <= 16'h0000;
    else if (IFDC_Flush)
        IFDC_reg <= 16'h0000;
    else if (IFDC_W)
        IFDC_reg <= Inst;
end

// RegFile
always@(posedge CLK)
begin

```

```

        if (EXWB_WB)
            RegFile[EXWB_WAddr] <= EXWB_WData;
        end

// operand 0 and 1 from register file
assign Reg0 = RegFile[IFDC_reg[9:5]];
assign Reg1 = RegFile[IFDC_reg[4:0]];

// decoder
always@(IFDC_reg[15:10])
begin
    case(IFDC_reg[15:10])
        6'b010000: begin
            DC_ALU_OP    = 4'b0000; //ADD
            DC_WB        = 1'b1;      //have writeback
            DC_DW        = 1'b0;      //no datawrite
            DC_LD        = 1'b0;      //no load
            DC_BJ        = 4'b0000; // no Branch, no JUMP
            PC_W         = 1'b1;      //LOW POWER PC WAIT
            IFDC_W       = 1'b1;      //LOW POWER IFDC WAIT
        end
        6'b010001: begin              //SUB
            DC_ALU_OP    = 4'b0001;
            DC_WB        = 1'b1;
            DC_DW        = 1'b0;
            DC_LD        = 1'b0;
            DC_BJ        = 4'b0000;
            PC_W         = 1'b1;
            IFDC_W       = 1'b1;
        end
        6'b010010: begin              //CMP
            DC_ALU_OP    = 4'b0001;
            DC_WB        = 1'b0;
            DC_DW        = 1'b0;
            DC_LD        = 1'b0;
            DC_BJ        = 4'b0000;
            PC_W         = 1'b1;
            IFDC_W       = 1'b1;
        end
        6'b010100: begin              //AND
            DC_ALU_OP    = 4'b0010;
            DC_WB        = 1'b1;
            DC_DW        = 1'b0;
            DC_LD        = 1'b0;
        end
    endcase
end

```

```

        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b010101: begin          //OR
        DC_ALU_OP  = 4'b0011;
        DC_WB      = 1'b1;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b010110: begin          //XOR
        DC_ALU_OP  = 4'b0100;
        DC_WB      = 1'b1;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b100000: begin          //LSL
        DC_ALU_OP  = 4'b0101;
        DC_WB      = 1'b1;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b100001: begin          //LSR
        DC_ALU_OP  = 4'b0110;
        DC_WB      = 1'b1;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b100010: begin          //ASR
        DC_ALU_OP  = 4'b0111;
        DC_WB      = 1'b1;
        DC_DW      = 1'b0;

```

```

        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end
    6'b100111: begin                //MOV
        DC_ALU_OP   = 4'b1000;
        DC_WB       = 1'b1;
        DC_DW       = 1'b0;
        DC_LD       = 1'b0;
        DC_BJ       = 4'b0000;
        PC_W        = 1'b1;
        IFDC_W      = 1'b1;
    end
    6'b011000: begin                //LD
        DC_ALU_OP   = 4'b1001;
        DC_WB       = 1'b1;
        DC_DW       = 1'b0;
        DC_LD       = 1'b1;
        DC_BJ       = 4'b0000;
        PC_W        = 1'b1;
        IFDC_W      = 1'b1;
    end
    6'b011001: begin                //ST
        DC_ALU_OP   = 4'b1010;
        DC_WB       = 1'b0;
        DC_DW       = 1'b1;
        DC_LD       = 1'b0;
        DC_BJ       = 4'b0000;
        PC_W        = 1'b1;
        IFDC_W      = 1'b1;
    end
    6'b110000: begin                //BZ
        DC_ALU_OP   = 4'b1011 ;
        DC_WB       = 1'b0;
        DC_DW       = 1'b0;
        DC_LD       = 1'b0;
        DC_BJ       = 4'b0001;
        PC_W        = 1'b1;
        IFDC_W      = 1'b1;
    end
    6'b110001: begin                //BNZ
        DC_ALU_OP   = 4'b1011 ;
        DC_WB       = 1'b0;

```

```

        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0010;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b110010: begin                //BLE
        DC_ALU_OP  = 4'b1011 ;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0011;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b110011: begin                //BLT
        DC_ALU_OP  = 4'b1011 ;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0100;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b110100: begin                //BGE
        DC_ALU_OP  = 4'b1011;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0101;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b110100: begin                //BGT
        DC_ALU_OP  = 4'b1011;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0110;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b110110: begin                //B
        DC_ALU_OP  = 4'b1011;

```

```

        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0111;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b111000: begin          //JUMP
        DC_ALU_OP  = 4'b1011;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b1000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b000000: begin          //NOP
        DC_ALU_OP  = 4'b1011;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b1;
        IFDC_W     = 1'b1;
    end

    6'b000001: begin          //WAIT
        DC_ALU_OP  = 4'b1011;
        DC_WB      = 1'b0;
        DC_DW      = 1'b0;
        DC_LD      = 1'b0;
        DC_BJ      = 4'b0000;
        PC_W       = 1'b0;
        IFDC_W     = 1'b0;
    end

endcase

end

// Branch

// select signal for PC
assign PC_Sel = (DC_BJ == 4'b0001 && ALU_Z == 1'b1)           ? 2'b01 : //BZ
                (DC_BJ == 4'b0010 && ALU_Z == 1'b0)           ? 2'b01 :
                //BNZ

```

```

                (DC_BJ == 4'b0011 && (ALU_Z == 1'b1 || ALU_N == 1'b1)) ? 2'b01 :
//BLE
                (DC_BJ == 4'b0100 && (ALU_Z == 1'b0 && ALU_N == 1'b1)) ? 2'b01 :
//BLT
                (DC_BJ == 4'b0101 && (ALU_Z == 1'b1 || ALU_N == 1'b0)) ? 2'b01 :
//BGE
                (DC_BJ == 4'b0110 && (ALU_Z == 1'b0 && ALU_N == 1'b0)) ? 2'b01 :
//BLT
                (DC_BJ == 4'b0111) ?
2'b01 : //BGE
                (DC_BJ == 4'b1000) ? 2'b10 :
//JUMP

2'b00; //NO_JUMP

// clear signal for IFDC register
assign IFDC_Flush = (PC_Sel != 2'b00) ? 1'b1 : 1'b0;

assign EX_BP_Sel_0 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr0)) ? 2'b01:
                (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr0)) ? 2'b10:
                2'b00;

assign EX_BP_Sel_1 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr1)) ? 2'b01:
                (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr1)) ? 2'b10:
                2'b00;

// bypass controller 0
assign DC_BP_Sel_0 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr0)) ? 2'b01:
                (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr0)) ? 2'b10:
                2'b00;

assign DC_BP_Sel_1 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr1)) ? 2'b01:
                (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr1)) ? 2'b10:
                2'b00;

// DataRAM interface
assign DW = DC_DW;
assign DAddr = (DC_BP_Sel_1 == 2'b01) ? DCEX_Reg1 :
                (DC_BP_Sel_1 == 2'b10) ? EXWB_Reg1 :
                Reg1;
assign WData = (DC_BP_Sel_0 == 2'b01) ? EXWB_WData :
                (DC_BP_Sel_0 == 2'b10) ? WBXX_WData :
                Reg0;

```



```

//
// EX stage
//

// DCEX registers
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn) begin
        DCEX_ALU_OP    <= 4'b0000;
        DCEX_DW        <= 1'b0;
        DCEX_LD        <= 1'b0;
        DCEX_WB        <= 1'b0;
        DCEX_Reg0      <= 32'h00000000;
        DCEX_Reg1      <= 32'h00000000;
        DCEX_imm       <= 5'h00;
        DCEX_RegAddr0  <= 5'h00;
        DCEX_RegAddr1  <= 5'h00;
    end
    else begin
        DCEX_ALU_OP    <= DC_ALU_OP;
        DCEX_DW        <= DC_DW;
        DCEX_LD        <= DC_LD;
        DCEX_WB        <= DC_WB;
        DCEX_Reg0      <= Reg0;
        DCEX_Reg1      <= Reg1;
        DCEX_imm       <= IFDC_reg[4:0];
        DCEX_RegAddr0  <= IFDC_reg[9:5];
        DCEX_RegAddr1  <= IFDC_reg[4:0];
    end
end

// bypass controller 1
assign EX_BP_Sel_0 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr0)) ? 2'b01:
                    (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr0)) ? 2'b10:
                    2'b00;

assign EX_BP_Sel_1 = (EXWB_WB && (EXWB_WAddr == DCEX_RegAddr1)) ? 2'b01:
                    (WBXX_WB && (WBXX_WAddr == DCEX_RegAddr1)) ? 2'b10:
                    2'b00;

```

```

// ALU

// operands
assign  ALU_OP0 = (EX_BP_Sel_0 == 2'b01) ? EXWB_WData :
              (EX_BP_Sel_0 == 2'b10) ? WBXX_WData :
              DCEX_Reg0;

assign  ALU_OP1 = (EX_BP_Sel_1 == 2'b01) ? EXWB_WData :
              (EX_BP_Sel_1 == 2'b10) ? WBXX_WData :
              DCEX_Reg1;

// ALU operations
always@(DCEX_ALU_OP or ALU_OP0 or ALU_OP1 or DCEX_imm or ALU_Z_reg or ALU_N_reg)
begin
    case(DCEX_ALU_OP)
        4'b0000: begin          //ADD
            ALU_Result = ALU_OP0 + ALU_OP1;
            if (ALU_Result == 0)
                ALU_Z = 1'b1;
            else
                ALU_Z = 1'b0;
            if (ALU_Result[32:31] == 2'b11)
                ALU_N = 1'b1;
            else
                ALU_N = 1'b0;
        end
        4'b0001:begin          //SUB、CMP
            ALU_Result = ALU_OP0 - ALU_OP1;
            if (ALU_Result == 0)
                ALU_Z = 1'b1;
            else
                ALU_Z = 1'b0;
            if (ALU_Result[32:31] == 2'b11)
                ALU_N = 1'b1;
            else
                ALU_N = 1'b0;
        end
        4'b0010:begin          //AND
            ALU_Result = ALU_OP0 & ALU_OP1;
            if (ALU_Result == 0)
                ALU_Z = 1'b1;
            else

```

```

        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
end
4'b0011:begin        //OR
    ALU_Result = ALU_OP0 | ALU_OP1;
    if (ALU_Result == 0)
        ALU_Z = 1'b1;
    else
        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
end
4'b0100:begin        //XOR
    ALU_Result = ALU_OP0 ^ ALU_OP1;
    if (ALU_Result == 0)
        ALU_Z = 1'b1;
    else
        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
end
4'b0101:begin        //LSL
    ALU_Result = {ALU_OP0[30:0],0};
    if (ALU_Result == 0)
        ALU_Z = 1'b1;
    else
        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
end
4'b0110:begin        //LSR
    ALU_Result = {0,ALU_OP0[31:1]};
    if (ALU_Result == 0)
        ALU_Z = 1'b1;
    else

```

```

        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
    end
4'b0111:begin        //ASR
    ALU_Result = {ALU_OP0[31],ALU_OP0[31:1]};
    if (ALU_Result == 0)
        ALU_Z = 1'b1;
    else
        ALU_Z = 1'b0;
    if (ALU_Result[32:31] == 2'b11)
        ALU_N = 1'b1;
    else
        ALU_N = 1'b0;
    end
4'b1000: begin        //MOV
    ALU_Result = {27'h00000000, DCEX_imm};
    ALU_Z = ALU_Z_reg;
    ALU_N = ALU_N_reg;
end
4'b1001: begin        //LD
    ALU_Result = 32'h00000000;
    ALU_Z = 1'b0;
    ALU_N = 1'b0;
end
4'b1010: begin        //ST
    ALU_Result = 32'h00000000;
    ALU_Z = 1'b0;
    ALU_N = 1'b0;
end
4'b1011: begin        //Bcc
    ALU_Result = 32'h00000000;
    ALU_Z = ALU_Z_reg;
    ALU_N = ALU_N_reg;
end
default: begin
    ALU_Result = ALU_OP1;
    ALU_Z = ALU_Z_reg;
    ALU_N = ALU_N_reg;
end
endcase
end
end

```

```

// flag registers
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn) begin
        ALU_Z_reg <= 1'b0;
        ALU_N_reg <= 1'b0;
    end
    else begin
        ALU_Z_reg <= ALU_Z;
        ALU_N_reg <= ALU_N;
    end
end

// writeback data in EX stage
assign EX_WData = (DCEX_LD) ? RData : ALU_Result[31:0];

//
// EXWB
//

// EXWB registers
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn) begin
        EXWB_LD      <= 1'b0;
        EXWB_WB      <= 1'b0;
        EXWB_Reg1     <= 32'h00000000;
        EXWB_WAddr    <= 5'h00;
        EXWB_WData    <= 32'h00000000;
    end
    else begin
        EXWB_LD      <= DCEX_LD;
        EXWB_WB      <= DCEX_WB;
        EXWB_Reg1     <= DCEX_Reg1;
        EXWB_WAddr    <= DCEX_RegAddr0;
        EXWB_WData    <= EX_WData;
    end
end

//
// WBXX
//

```

```
// WBXX registers
always@(posedge CLK or negedge Rstn)
begin
    if (!Rstn) begin
        WBXX_WB      <= 1'b0;
        WBXX_WAddr   <= 5'h00;
        WBXX_WData   <= 32'h00000000;
    end
    else begin
        WBXX_WB      <= EXWB_WB;
        WBXX_WAddr   <= EXWB_WAddr;
        WBXX_WData   <= EXWB_WData;
    end
end

endmodule
```

2.代码解释

PC 更新选择逻辑

```
assign PC_addoffset = PC_reg + {{21{!IFDC_reg[9]}},IFDC_reg[9:0],1'b0};

assign PC = (Rstn_flag == 1'b1)? Rstn_Addr :
            (PC_Sel == 2'b00)? PC_add2 :
            (PC_Sel == 2'b01) ? PC_addoffset :
            {21'h000000,IFDC_reg[9:0],1'b0};
```

PC 选择逻辑

PC 表示下一条指令的地址,它由复位状态和 PC_Sel 共同决定,属于一个多路选择器结构:

- 复位优先: 若 Rstn_flag==1,处理器处于复位进入阶段,PC 被强制设置为 Rstn_Addr,保证从固定入口开始执行程序。
- 正常顺序执行: 当 PC_Sel==2'b00,说明没有分支/跳转发生,PC 按指令对齐方式正常递增,即 PC = PC_add2 (PC+2)。
- 分支成立: 当 PC_Sel==2'b01,说明某条分支条件满足,需要跳转到相对目标地址,PC 取 PC_addoffset (PC + offset)。
- 跳转指令 (JUMP): 当 PC_Sel==2'b10,PC 取绝对目标地址 {21'h000000, IFDC_reg[9:0], 1'b0},即由指令中的立即数字段直接组成跳转目的地址。

偏移相对地址计算 (有符号数)

PC_addoffset 用于计算分支类指令的相对目标地址

取出偏移字段: IFDC_reg[9:0] 为指令中携带的偏移量;使用最高位 IFDC_reg[9] 作为符号位,把 10 位偏移扩展成 32 位有符号数,保证偏移既可为正也可为负;对齐处理 (×2):再拼接 1'b0 使偏移左移 1 位,相当于乘以 2。

绝对地址计算 (无符号数)

用于 JUMP 跳转指令的绝对地址生成

IFDC_reg[9:0] 被当作一个无符号地址字段使用;高位用 21'h000000 填 0,相当于把 10 位立即数扩展成 32 位无符号数;低位补 1'b0,同样用于 2 字节对齐。

取指流水线冲刷

```
assign PC_Sel = (DC_BJ == 4'b0001 && ALU_Z == 1'b1) ? 2'b01 :    //BZ
                (DC_BJ == 4'b0010 && ALU_Z == 1'b0) ? 2'b01 :    //BNZ
                (DC_BJ == 4'b0011 && (ALU_Z == 1'b1 || ALU_N == 1'b1)) ? 2'b01 : //BLE
                (DC_BJ == 4'b0100 && (ALU_Z == 1'b0 && ALU_N == 1'b1)) ? 2'b01 : //BLT
                (DC_BJ == 4'b0101 && (ALU_Z == 1'b1 || ALU_N == 1'b0)) ? 2'b01 : //BGE
                (DC_BJ == 4'b0110 && (ALU_Z == 1'b0 && ALU_N == 1'b0)) ? 2'b01 : //BGT
                (DC_BJ == 4'b0111) ? 2'b01 :    //B
                (DC_BJ == 4'b1000) ? 2'b10 : 2'b00;    //JUMP

// clear signal for IFDC register
assign IFDC_Flush = (PC_Sel != 2'b00) ? 1'b1 : 1'b0;
```

PC_Sel 由译码阶段给出的分支/跳转类型信号 DC_BJ，结合 ALU 产生的标志位 ALU_Z（零标志）与 ALU_N（负标志）共同决定下一条 PC 的取值方向。具体而言：当满足相应条件分支（BZ、BNZ、BLE、BLT、BGE、BGT 以及无条件分支 B）的触发条件时，PC_Sel 置为 2'b01，处理器转向分支目标地址；当指令为 JUMP 时，PC_Sel 置为 2'b10，PC 转向绝对跳转目标地址；若不满足任何跳转条件，则 PC_Sel 保持为 2'b00，PC 按顺序执行方式更新为 PC+2。

IFDC_Flush 作用：

IFDC_Flush 用于处理分支，跳转产生的控制冒险。当 PC_Sel != 2'b00 时说明 PC 将跳转，IF 阶段可能已取到错误的顺序下一条指令，因此置 IFDC_Flush=1 清空 IF/DC 流水寄存器，避免错误指令进入后续阶段，保证执行路径正确。

写回数据选择

```
assign EX_WData = (DCEX_LD) ? RData : ALU_Result[31:0];
```

该语句在 EX 阶段生成统一的写回数据 EX_WData：

若 DCEX_LD=1（加载指令 LD），写回数据应来自数据存储器读出值 RData；

若 DCEX_LD=0（算术逻辑、移位、MOV 等），写回数据来自 ALU 结果 ALU_Result[31:0]。

这样后续写回级无需再区分指令类型，统一使用 EX_WData 即可完成寄存器写回，数据通路更清晰。

DC 阶段数据旁路选择

```
assign DC_BP_Sel_0 = ((IFDC_reg[9:5] == DCEX_RegAddr0) && DCEX_WB)?2'b01:
                    ((IFDC_reg[9:5] == EXWB_WAddr) && EXWB_WB)?2'b10:2'b00;
assign DC_BP_Sel_1 = ((IFDC_reg[4:0] == DCEX_RegAddr0) && DCEX_WB)?2'b01:
                    ((IFDC_reg[4:0] == EXWB_WAddr) && EXWB_WB)?2'b10:2'b00;
assign DAddr      = (DC_BP_Sel_1 == 2'b00) ? Reg1 :
                    (DC_BP_Sel_1 == 2'b01) ? EX_WData:
                    EXWB_WData;
assign WData      = (DC_BP_Sel_0 == 2'b00) ? Reg0 :
                    (DC_BP_Sel_0 == 2'b01) ? EX_WData :
                    EXWB_WData;
```

在 DC 阶段，处理器会比较当前指令要读取的源寄存器编号（IFDC_reg[9:5] 或 IFDC_reg[4:0]）与前面流水线中指令的写回目的寄存器编号（如 DCEX_RegAddr0、EXWB_WAddr）。如果两者相等，并且前面那条指令确实会写回寄存器（DCEX_WB 或 EXWB_WB 为 1），就说明当前指令需要的数据还没写回到寄存器堆，直接读寄存器会得到旧值，因此必须启用旁路。

旁路选择信号 DC_BP_Sel_0/1 按照由近到远的原则进行选择：

- 1) 优先检查 DCEX 级，若发生相关则选择 2' b01，数据来自 EX_WData；
- 2) 若 DCEX 级不相关，再检查 EXWB 级，相关则选择 2' b10，数据来自 EXWB_WData；
- 3) 如果两级都不相关，则选择 2' b00，直接使用寄存器堆输出 Reg0/Reg1。

如果上一条和上上条都相关，当两条前序指令都要写同一个寄存器、且当前指令也要读这个寄存器时，系统会优先选择更近的 DCEX 级结果。原因是 DCEX 级代表最新产生的值，比 EXWB 级更接近当前时刻，能够保证使用的是最新、正确的数据，从而避免用到被覆盖的旧结果。

EX 阶段数据旁路选择

```
assign EX_BP_Sel_0 = ((DCEX_RegAddr0 == EXWB_WAddr) && EXWB_WB)?2'b01:
                    ((DCEX_RegAddr0 == WBXX_WAddr) && WBXX_WB)?2'b10:2'b00;
assign EX_BP_Sel_1 = ((DCEX_RegAddr1 == EXWB_WAddr) && EXWB_WB)?2'b01:
                    ((DCEX_RegAddr1 == WBXX_WAddr) && WBXX_WB)?2'b10:2'b00;
assign ALU_OP0 = (EX_BP_Sel_0 == 2'b00) ? {DCEX_Reg0[31], DCEX_Reg0} :
                (EX_BP_Sel_0 == 2'b01) ? {EXWB_WData[31], EXWB_WData} :
                {WBXX_WData[31], WBXX_WData};
assign ALU_OP1 = (EX_BP_Sel_1 == 2'b00) ? {DCEX_Reg1[31], DCEX_Reg1} :
                (EX_BP_Sel_1 == 2'b01) ? {EXWB_WData[31], EXWB_WData} :
                {WBXX_WData[31], WBXX_WData};
```

在 EX 阶段，ALU 即将使用的两个操作数分别来自 DCEX_Reg0/Reg1，其对应的源寄存器编号为 DCEX_RegAddr0 和 DCEX_RegAddr1。为了避免数据冒险，处理器会将这两个源寄存器编号与后续流水级中即将写回的目的寄存器编号进行比较（如 EXWB_WAddr、WBXX_WAddr）。

如果 寄存器地址相同，并且对应的指令确实会写回（EXWB_WB 或 WBXX_WB 为 1），就说明当前 EX 阶段需要的数据尚未写回到寄存器堆，直接使用 DCEX_Reg0/Reg1 会得到旧值，因此必须启用旁路，改用更新后的最新结果。

旁路选择信号 EX_BP_Sel_0/1 按照由近到远的原则进行选择：

- 1) 优先检查 EXWB 级，若发生相关，则选择 2' b01，操作数来自 EXWB_WData；
- 2) 若 EXWB 级不相关，再检查 WBXX 级，若发生相关，则选择 2' b10，操作数来自 WBXX_WData；
- 3) 若两级都不相关，则选择 2' b00，直接使用 DCEX_Reg0/Reg1 作为 ALU 输入。

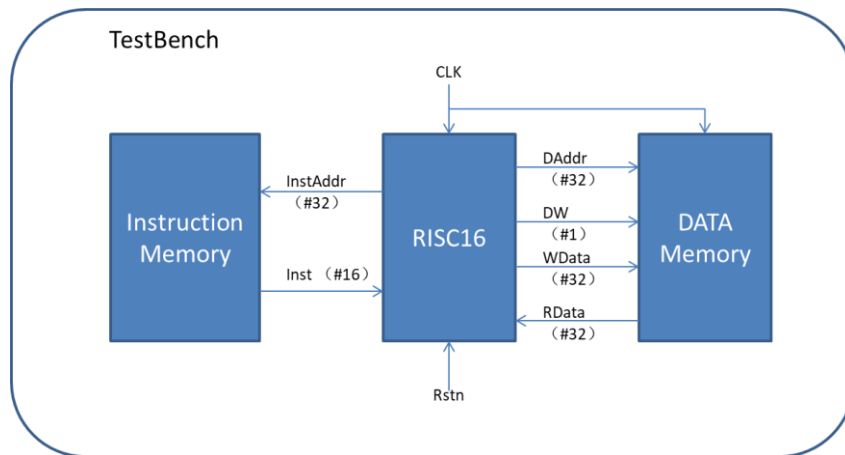
因此，ALU_OP0/ALU_OP1 会根据选择信号从 DCEX_Reg、EXWB_WData、WBXX_WData 三者中选取最合适的数据源，从而保证 ALU 始终拿到正确操作数。

当出现连续写同一寄存器的情况（例如上一条、上上条都写同一个寄存器，而当前指令在 EX 阶段要读取它），两级比较可能同时命中。此时系统会优先选择更近的 EXWB 级结果。原因是 EXWB 级代表距离当前时刻更近、更新更晚的结果，也就是最新值，能够避免取到 WBXX 中较早的旧结果，从而保证运算正确性与流水线连续执行。

四、测试环境、测试结果及分析

1.Modelsim 仿真

仿真平台



步骤

1. 完成/RISC16/RISC16.v 的 RTL 设计;
2. 在 simulation 中建立 ModelSim 工程，添加设计文件 (RISC16.v) 及测试文件 (RISC16_test.v, InstrAM.v, DataRAM.v)，编译;
3. 在脚本窗口输入 vsim work.RISC16_test
4. 执行仿真: run -all

如果结果正确，会显示消息 “Simulation completely successful”，否则显示消息 “Execution error happened!!!”。

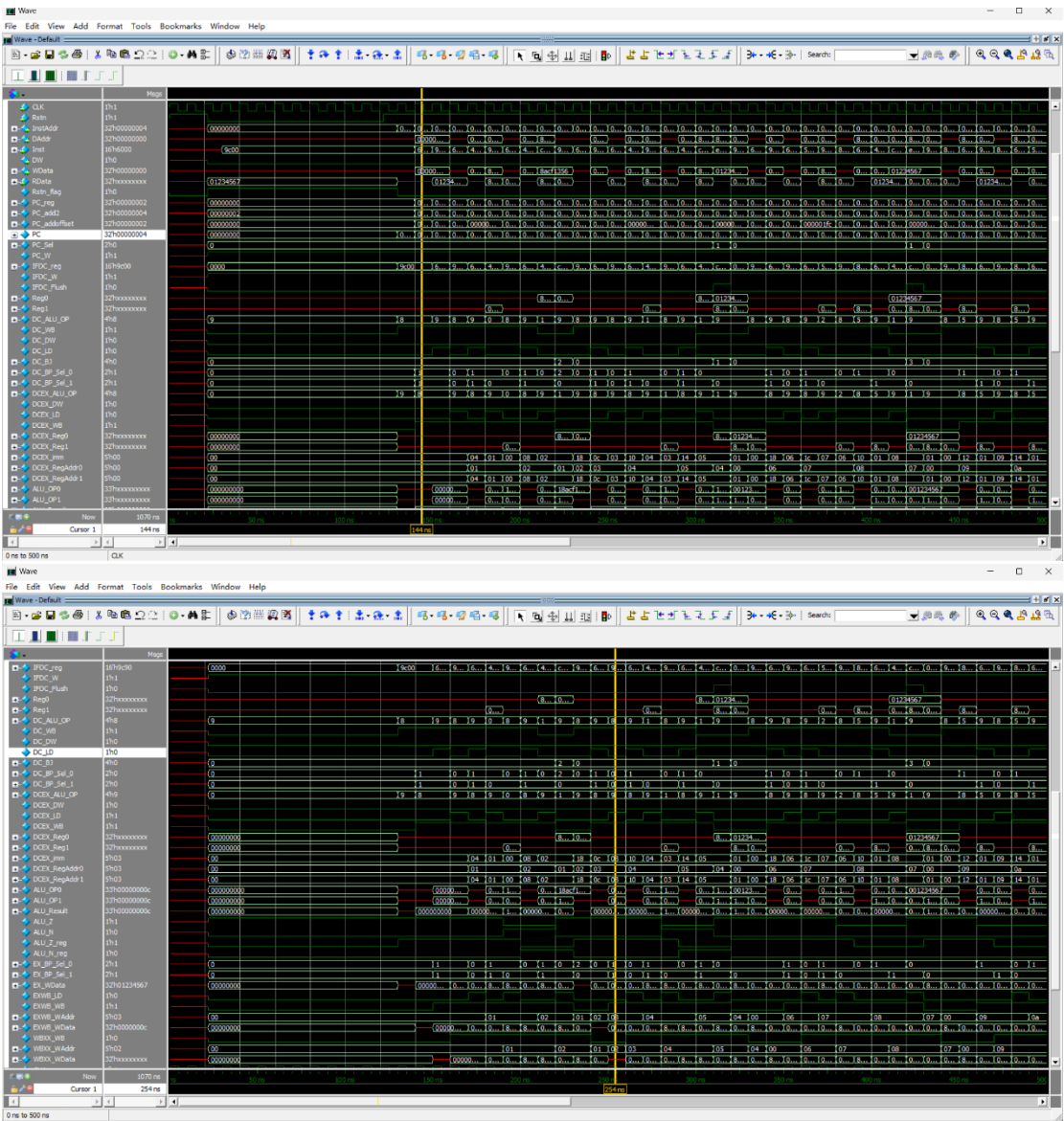
重新执行: restart

仿真结果

```
# Loading project edecpu
# Compile of DataRAM.v was successful.
# Compile of InstRAM.v was successful.
# Compile of RISCie.v was successful.
# Compile of RISCie_test.v was successful.
# Compile of RISCie.v failed with no errors.
ModelSim>vaim work.RISCie_test
# Start time: 16:25:16 on Sun 21,2024
# ** Note: (vaim-3112) Design is being optimized...
# ** Warning: D:/Users/66187/Desktop/EDCpu/simulation/RISCie_test.v(27): (vspt-2241) Connection width does not match width of port 'DAddr'. The port definition is at: D:/Users/66187/Desktop/EDCpu/simulation/DataRAM.v(3).
# ** Warning: D:/Users/66187/Desktop/EDCpu/simulation/RISCie_test.v(37): (vspt-2241) Connection width does not match width of port 'InstAddr'. The port definition is at: D:/Users/66187/Desktop/EDCpu/simulation/InstRAM.v(3).
# Loading work.RISCie_test(fast)
VSNM>run -all
# Simulation completely successful
```

按照步骤运行后，可以看到显示消息“Simulation completely successful”，意味着仿真过程顺利执行。

波形分析



```

InstRAM.dat
文件 编辑 查看

100111_00000_00000 //MOV r0 0
011000_00000_00000 //LD r0 r0
100111_00001_00100 //MOV r1 4
011000_00001_00001 //LD r1 r1
010000_00001_00000 //ADD r1 r0
100111_00010_01000 //MOV r2 8
011000_00010_00010 //LD r2 r2
010010_00001_00010 //CMP r1 r2
110001_0001011000 //BNZ ERROR-- 88
100111_00011_01100 //MOV r3 12
011000_00011_00011 //LD r3 r3
100111_00100_10000 //MOV r4 16
011000_00100_00100 //LD r4 r4
010001_00100_00011 //SUB r4 r3
100111_00101_10100 //MOV r5 20
011000_00101_00101 //LD r5 r5
010010_00100_00101 //CMP r4 r5
110000_0000000001 //BZ test_AND -- 1
111000_0001100001 //JUMP ERROR -- 97
100111_00110_11000 //test_AND: MOV r6 24
011000_00110_00110 //LD r6 r6
100111_00111_11100 //MOV r7 28
011000_00111_00111 //LD r7 r7
010100_00111_00110 //AND r7 r6
100111_01000_10000 //MOV r8 16
100000_01000_00001 //LSL r8 1
011000_01000_01000 //LD r8 r8

```

进一步从波形验证指令级行为，在第一个周期的 IA 阶段，处理器主要完成取指地址的产生与更新，即确定本次将要访问的指令存储器地址；随后进入 IF 阶段完成取指并将指令送入流水线。波形中 Inst 显示为 011000_00000_00000，对应十六进制 16'h6000，与指令集定义一致，可译为 LD r0, r0，表示对寄存器 r0 相关的数据进行加载并写回到 r0。该现象说明取指结果正确、指令编码与显示一致。

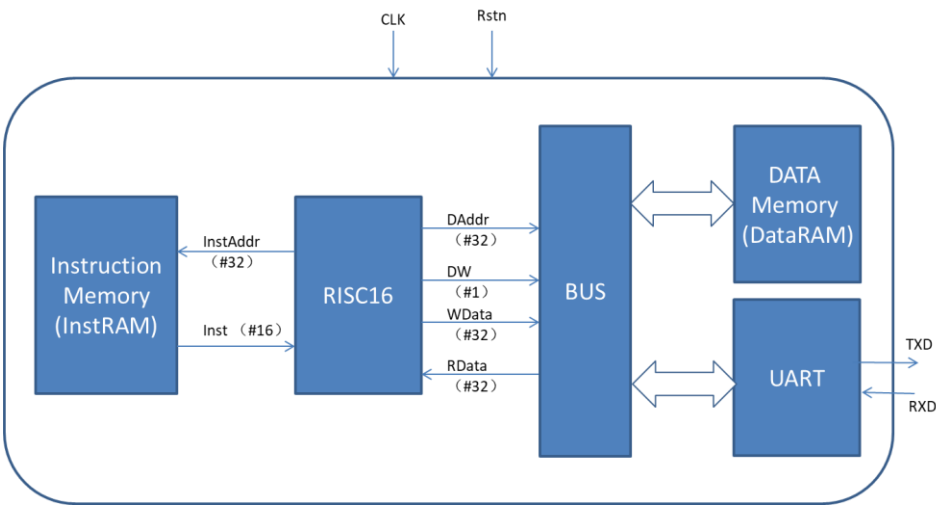
进入 DC 阶段后，可以结合 IF/DC 流水寄存器与译码逻辑进行对应验证，设计中 IFDC_reg 用于保存 IF→DC 阶段传递的指令，并由 IFDC_W 控制是否写入。对指令 16'h6000，译码分支会产生一组明确且稳定的控制信号：DC_ALU_OP=4'b1001、DC_WB=1、DC_LD=1、DC_DW=0、PC_W=1、IFDC_W=1，该指令属于加载（LD）类型，需要在后续阶段完成数据读取并写回寄存器（WB=1）；同时不进行数据存储写 RAM（DW=0），PC 允许正常更新（PC_W=1）。波形中控制信号与译码设定相吻合，说明译码结果与指令语义一致。

EX 阶段，DC 阶段产生的控制信号与操作数通过 DCEX 流水寄存器传递至执行级，包括 DCEX_ALU_OP、DCEX_LD/DCEX_WB/DCEX_DW 以及 DCEX_Reg0/DCEX_Reg1 等。波形中可观察到 DCEX_LD=1、DCEX_WB=1、DCEX_DW=0 与 DC 阶段保持一致，同时 ALU 输入 ALU_OP0/ALU_OP1 以及旁路选择 EX_BP_Sel_0/1 工作正常，说明控制信号与数据通路在流水级之间传递正确，执行级能够按照 LD 指令路径完成必要的地址/数据相关运算，并为后续写回准备正确的数据。

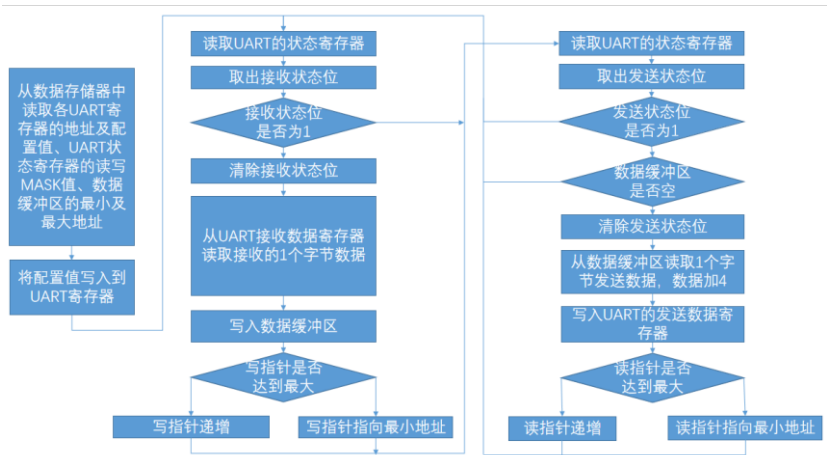
WB 阶段，写回相关信息由 EXWB 流水寄存器承接：EXWB_WB 表示写回使能，EXWB_WAddr 为目的寄存器地址，EXWB_WData 为最终写回数据。波形显示写回使能有效、写回地址指向 r0，并且 r0 在对应时钟沿完成更新，从而形成了完整闭环，证明该指令在流水线中的执行链路是连续、正确且符合预期的。

2.FPGA 测试（RISC16+UART 通信）

RISC16+UART 通信平台



程序流程图



测试步骤

1. 在 FPGA 子目录中建立 QuartusII 工程 RISC16_UART;
2. 利用 MegaWizard Plug-in Manager 定制 InstRAM(1-port:ROM, 256x16, 输出端口不用寄存器保存), 并指定初始化文件 InstRAM.hex.
3. 利用 MegaWizard Plug-in Manager 定制 DataRAM(1-port:RAM, 256x32, 输出端口不用寄存器保存), 并指定初始化文件 DataRAM.hex.
4. 添加所有设计文件 (RISC16, UART, BUS, InstRAM, DataRAM);
5. 设计顶层模块 RISC16_UART (代码方式或原理图方式)
6. 引脚分配;
7. 在 Assignment>Setting 中, 勾选 Physical Synthesis Optimization>Perform register retiming 及 Effort level:Extra; Analysis & Synthesis Setting>Optimization Technique:Speed, 进行编译。
8. 配置 FPGA, 连接 RS232 电缆, 运行串口通信.exe, 设置串口 (设备管理器中可查找串口号), 波特率 4800, 数据位 8, 校验位无, 停止位 1, 不勾选校验, 勾选 HEX、自动换行、显示间隔等; 点击“打开串口”, 在发送栏输入数据“00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F”, 点击发送, 观察接受窗口是否能够接受到每个字节+4 后结果。

测试结果与分析

串口通信结果如图所示, 接收窗口返回的数据与预期一致: 每个输入字节都被正确执行了“+4”操作后返回, 且回传顺序与发送顺序对应, 无错位、无异常字符。



五、总结

本次 EDA 课程设计围绕基于 Verilog HDL 的 RISC16 五级流水线处理器的设计与实现展开，完整覆盖了从指令集架构定义、处理器体系结构规划、数据通路与控制通路设计，到流水线机制实现、冒险处理策略集成，再到功能仿真验证与 FPGA 平台测试的全流程。与以往模块化的验证实验相比，本次设计在系统性、复杂度和工程实现层面提出了更高的要求，是一次对硬件系统设计能力的综合锻炼与全面提升。

面对从零开始构建一个 CPU 内核的实践任务，我深刻体会到理论知识与工程实现之间的衔接挑战。尽管在相关课程中已掌握了流水线基本概念与指令执行流程，但在具体实现中，如何合理划分流水级、规范信号传递、确保时序一致，仍需反复推敲与实践验证。尤其是在五级流水线的级间协同、控制信号对齐以及数据旁路转发机制的设计过程中，我对处理器内部并行执行、时序约束及相关性处理有了更为具体和深入的理解。

在体系结构设计阶段，通过反复研读课程资料、分析代码框架，并结合结构图与信号流向图的绘制，我逐步明确了 IA、IF、DC、EX、WB 各阶段的功能划分与接口定义。这一过程让我认识到，清晰合理的架构设计是实现可靠性与可扩展性的基础。特别在引入 WBXX 暂存级以扩展旁路覆盖范围后，我对“以硬件资源换取执行效率”的流水线设计思想有了更真切的体会。

对于编码实现与调试，我着重对各类指令的执行路径进行逐条梳理与波形跟踪。通过仿真工具观察指令在流水线中的推进状态，系统理解了算术逻辑、数据访存及控制转移指令在数据通路中的具体行为。在实现数据冒险处理时，通过设计两级旁路（EX→EX、WB→EX）并将其扩展至访存地址与写数据通路，有效缓解了 RAW 相关导致的流水线停顿；在控制冒险处理方面，通过 PC 选择逻辑与 IFDC 冲刷机制，确保了分支与跳转指令执行路径的正确性。这些机制的实现，不仅提升了处理器的执行效率，也深化了我对流水线处理器“正确性优先，效率优化”设计原则的理解。

在功能验证环节，我首先借助 ModelSim 仿真平台对处理器进行了全面测试，验证了其数据通路、控制逻辑及冒险处理机制的正确性；随后在 FPGA 平台上搭建了包含 UART 通信的完整片上系统，实现了从指令加载、程序执行到串口输出的全过程硬件验证。这一过程不仅锻炼了我的系统集成与调试能力，也加深了我对 CPU 与存储器、外设之间协同工作方式的认知。

回顾整个课程设计历程，我从对处理器实现细节一知半解的状态，逐步成长为能够独立完成流水线 CPU 核心设计、具备初步系统调试能力的实践者。我深切认识到，高质量的硬件设计不仅依赖于扎实的理论基础，更离不开清晰的系统思维、严谨的工程习惯以及耐心细致的问题分析与解决能力。在此过程中，与同学们的讨论交流、老师的及时指导也为克服难点提供了重要支持。

当然，本设计仍有进一步优化与拓展的空间，例如可引入动态分支预测机制以减少控制冒险开销、添加异常与中断处理以增强处理器的实用性与鲁棒性、扩展指令集与存储层次以提升系统整体性能等。这些方向既是后续学习的重点，也为我在计算机体系结构及硬件设计领域的持续探索指明了路径。

最后，衷心感谢张俊老师在 EDA 课程教学与课程设计指导中给予的悉心教诲与无私帮助，使我在面对复杂系统设计时能够稳步推进、持续成长。本次课程设计不仅强化了我对流水线处理器与 RISC 架构的理解，更显著提升了我的工程实践能力与系统级设计思维，为我未来在 FPGA 与计算机硬件设计领域的深入学习与研究奠定了扎实的基础。